

MedBrain

Arquitetura Técnica do MVP

Stack, estrutura de pastas
e fluxos principais

Equipe de desenvolvimento • MVP

Versão 1.0

Sumário

1. Sobre este Documento
2. Visão Geral da Arquitetura
3. Stack de Tecnologias
4. Estrutura de Pastas
 - 4.1 Frontend (Next.js)
 - 4.2 Backend (Nest.js)
5. Fluxos Principais
 - 5.1 Autenticação
 - 5.2 Upload de PDF e Geração de Conteúdo
 - 5.3 Estudo (Flashcards e Questões)
 - 5.4 Ranking Semanal
6. Módulo de IA
7. Banco de Dados (Prisma + PostgreSQL)
8. Infraestrutura e Deploy
9. Convenções de Código
10. O que NÃO entra agora

1. Sobre este Documento

Este documento descreve a **arquitetura técnica** do MedBrain no MVP. Ele complementa o documento de *Visão Geral e Regras de Negócio* e deve ser lido em seguida. O foco aqui é orientar os 1–2 desenvolvedores full-stack que vão tocar o projeto, garantindo que decisões de pasta, módulo, dependência e fluxo estejam alinhadas.

Premissas técnicas

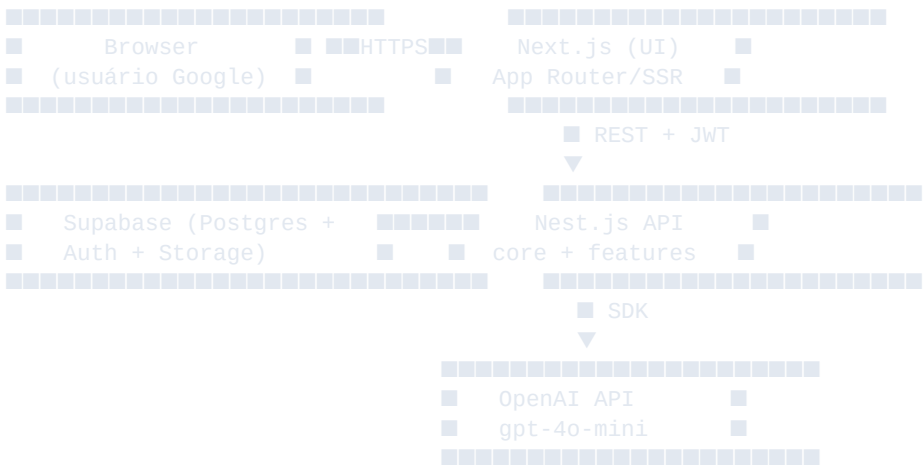
- Time pequeno (1–2 full-stack), então a arquitetura deve ser **simples e direta**.
- **Sem créditos no MVP**: a geração de IA é livre, mas o módulo placeholder fica preparado para evolução futura.
- Reuso de conteúdo via **banco centralizado por tópico** é parte da arquitetura, não opcional.
- Padrão de referência: projeto **MyMobiConf**, com separação core/ + features/.

2. Visão Geral da Arquitetura

A aplicação é composta por **três camadas independentes** que se comunicam por HTTP e dependem de serviços gerenciados do Supabase:

- **Frontend**: SPA renderizada com Next.js (App Router), responsável por toda a experiência do usuário.
- **Backend**: API REST em Nest.js, responsável pelas regras de negócio, integração com OpenAI e acesso ao banco via Prisma.
- **Banco e Auth**: PostgreSQL hospedado no Supabase, com Supabase Auth para Google OAuth. O Nest valida o JWT emitido pelo Supabase.

Diagrama lógico (resumo)



3. Stack de Tecnologias

Frontend framework	Next.js (App Router)
UI components	Shadcn/ui + Tailwind CSS
Animações	GSAP
Formulários	React Hook Form + Zod
Backend framework	Nest.js
ORM	Prisma
Banco de dados	PostgreSQL (hospedado no Supabase)
Autenticação	Supabase Auth (Google OAuth) — JWT validado pelo Nest
IA	OpenAI SDK oficial, modelo gpt-4o-mini
Gerenciador de pacotes	pnpm (em todos os pacotes)
Containerização	Docker + Docker Compose

Decisões técnicas explícitas

- **Sem frameworks de orquestração de IA** (sem LangChain, sem Agno). Apenas chamadas diretas ao SDK da OpenAI para manter simplicidade e custo previsíveis.
- **Supabase para Auth e DB**, mas o backend é Nest.js — o Supabase atua como infra gerenciada, não como backend completo.
- **Banco centralizado de conteúdo por tópico** (não por usuário), o que é decisão de arquitetura, não detalhe de implementação.
- **Módulo de pagamento existe apenas como placeholder**: pastas e contratos prontos, mas sem lógica real no MVP.

4. Estrutura de Pastas

A estrutura segue o padrão do projeto-referência **MyMobiConf**: separação clara entre core/ (infra compartilhada e cross-cutting) e features/ (domínio do produto, uma pasta por feature).

4.1 Frontend (Next.js)

```
medbrain-frontend/
├── app/                                # App Router (rotas)
│   ├── (auth)/login/
│   ├── (app)/
│   │   ├── dashboard/
│   │   ├── upload/
│   │   ├── flashcards/[id]/
│   │   ├── questions/[id]/
│   │   ├── groups/
│   │   └── layout.tsx
│   └── src/
│       ├── core/                      # infra compartilhada
│       │   ├── api/                  # cliente HTTP (fetch wrapper)
│       │   ├── auth/                 # hooks de sessão Supabase
│       │   ├── ui/                   # componentes shadcn re-exportados
│       │   ├── animations/           # presets GSAP
│       │   ├── lib/                  # utils, formataadores
│       │   └── features/
│       │       ├── upload/            # form de upload + serviços
│       │       ├── flashcards/        # estudo de flashcards
│       │       ├── questions/         # estudo de questões
│       │       ├── groups/            # criação/entrada por convite
│       │       ├── ranking/           # ranking semanal
│       │       └── dashboard/         # widgets do dashboard
│       ├── public/
│       ├── tailwind.config.ts
│       ├── package.json
│       └── pnpm-lock.yaml
```

4.2 Backend (Nest.js)

```
medbrain-backend/
├── prisma/
│   ├── schema.prisma
│   └── migrations/
├── src/
│   ├── core/
│   │   ├── auth/                    # guard JWT (Supabase)
│   │   ├── database/                # PrismaService
│   │   ├── ai/                      # cliente OpenAI + prompts
│   │   ├── config/                  # env, validação Zod
│   │   ├── filters/                 # exceptions globais
│   │   ├── interceptors/            # logging, transformação
│   │   └── features/
│   │       ├── users/
│   │       ├── groups/
│   │       ├── invites/
│   │       ├── pdfs/                # upload + parsing
│   │       ├── content/             # banco centralizado por tópico
│   │       ├── flashcards/
│   │       ├── questions/
│   │       ├── answers/             # histórico de respostas
│   │       ├── ranking/             # cálculo + reset semanal
│   │       └── billing/             # PLACEHOLDER (sem lógica)
│   ├── app.module.ts
│   ├── main.ts
│   └── Dockerfile
```

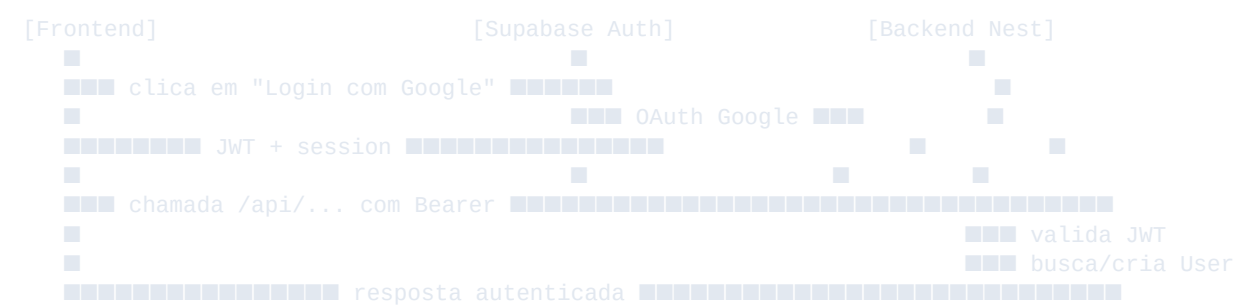
```
■■■ docker-compose.yml
■■■ package.json
■■■ pnpm-lock.yaml
```

Regra de ouro da estrutura

- **core/** contém o que é compartilhado e estável (infra, auth, banco, IA).
- **features/** contém domínio. Cada feature tem seu próprio módulo, controller, service, dto e — quando faz sentido — schema.
- Features **não importam diretamente** de outras features. Se precisarem se comunicar, fazem via serviço exposto explicitamente ou via core.

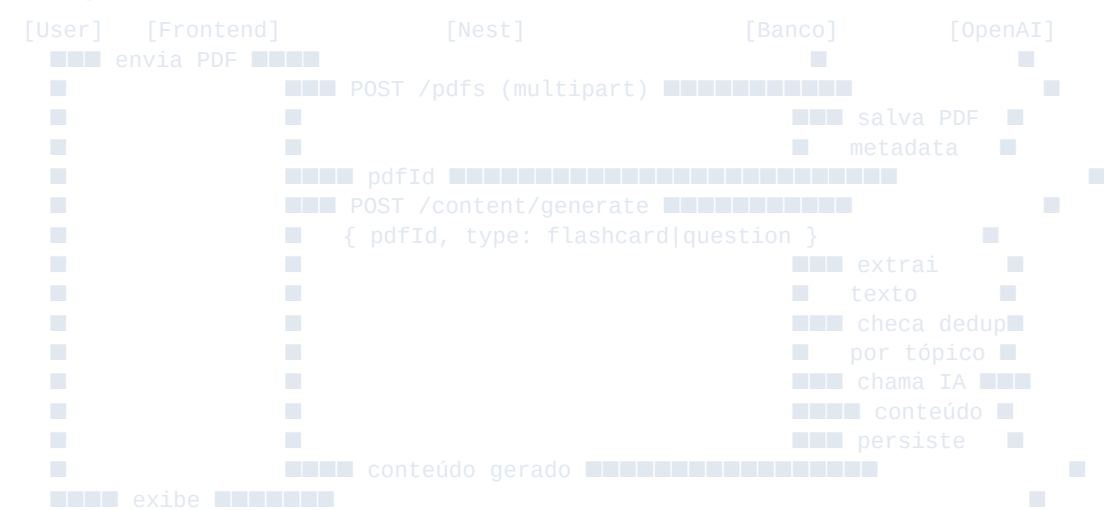
5. Fluxos Principais

5.1 Autenticação



- O frontend nunca toca diretamente no banco — sempre passa pelo Nest.
- O Nest **valida o JWT do Supabase** em todo endpoint protegido via guard.
- Na primeira autenticação, o Nest cria o registro User no banco.

5.2 Upload de PDF e Geração de Conteúdo



- O **parsing do PDF** acontece no backend (sem dependência do frontend).
- Antes de chamar a IA, o backend **verifica duplicidade por tópico** no banco centralizado.
- Se já existe conteúdo equivalente, o sistema reaproveita em vez de gerar novamente.
- No MVP, **não há checagem de créditos** — o módulo placeholder apenas registra que houve consumo, para uso futuro.

5.3 Estudo (Flashcards e Questões)

- **Flashcards:** o frontend exibe frente; ao clicar/virar, mostra verso. O usuário marca se acertou ou errou (somente para estatística, sem pontuação no MVP).
- **Questões:** o frontend mostra enunciado + 4 alternativas. Ao responder, envia POST /answers com a alternativa escolhida.
- O backend valida a resposta, registra no histórico do usuário e, se for acerto, incrementa a pontuação semanal do usuário no grupo correspondente.
- A explicação só é revelada **após** o usuário responder.

5.4 Ranking Semanal

- Cada acerto soma pontos em um registro WeeklyScore(userId, groupId, weekStart).
- O endpoint GET /groups/:id/ranking retorna o ranking da semana vigente ordenado por pontos (desempate por questões respondidas).
- O **reset é implícito:** a chave de semana muda automaticamente toda segunda-feira, então registros antigos deixam de aparecer no ranking sem precisar deletá-los.
- Medalhas (top 3) são **derivadas** da consulta, não armazenadas como evento separado no MVP.

6. Módulo de IA

Toda a lógica de IA fica em `src/core/ai/`. Não há orquestrador externo: o módulo expõe funções tipadas que recebem texto de entrada e devolvem objetos validados com Zod.

Contrato esperado

```
// src/core/ai/ai.service.ts
@Injectable()
export class AIService {
  constructor(private readonly openai: OpenAI) {}

  async generateFlashcards(text: string): Promise<Flashcard[]> {
    const res = await this.openai.chat.completions.create({
      model: 'gpt-4o-mini',
      response_format: { type: 'json_object' },
      messages: [
        { role: 'system', content: FLASHCARD_SYSTEM_PROMPT },
        { role: 'user', content: text },
      ],
    });
    return FlashcardArraySchema.parse(JSON.parse(res.choices[0].message.content!));
  }

  async generateQuestions(text: string): Promise<Question[]> {
    // mesma ideia, com QuestionArraySchema (4 alternativas, 1 correta, explicação)
  }
}
```

- Toda resposta do modelo passa por **validação Zod** antes de chegar à camada de negócio. Saídas inválidas são rejeitadas com `retry` limitado.
- Prompts ficam em arquivos separados (`prompts/flashcards.ts`, `prompts/questions.ts`) para facilitar versionamento.
- **Sempre 4 alternativas, 1 correta, com explicação** — isso é parte do schema, não opcional.

7. Banco de Dados (Prisma + PostgreSQL)

O banco fica no Supabase. As principais entidades do MVP são:

User	Identidade do usuário, vinculada ao Supabase Auth.
Group	Grupos privados de estudo.
GroupMember	Relação N:N entre usuários e grupos.
Invite	Links de convite para entrar em grupos.
Pdf	Metadados do PDF enviado (não o binário em si).
Topic	Tópico/assunto que organiza o banco centralizado de conteúdo.
Flashcard	Cartão (frente/verso) vinculado a um Topic.
Question	Questão objetiva (4 alternativas, 1 correta, explicação).
Answer	Registro de cada resposta dada por um usuário.

WeeklyScore	Pontuação do usuário em um grupo, por semana.
--------------------	---

Regras estruturais

- **Flashcard** e **Question** referenciam **Topic**, não **User**. Isso é o que viabiliza o banco centralizado.
- **Answer** referencia `userId`, `questionId` e `groupId`.
- **WeeklyScore** usa chave composta (`userId`, `groupId`, `weekStart`) para permitir reset implícito por semana.
- Toda tabela tem `createdAt` e `updatedAt`.

8. Infraestrutura e Deploy

Frontend deploy	Vercel (Next.js).
Backend deploy	Container Docker (provedor a definir — Railway, Fly.io ou similar).
Banco	Supabase (PostgreSQL gerenciado).
Auth	Supabase Auth (Google OAuth).
Variáveis sensíveis	Em .env, fora do versionamento, replicadas no provedor.
Dev local	Docker Compose levanta backend; banco aponta para Supabase de dev.

Variáveis de ambiente essenciais

```
# Backend
DATABASE_URL=postgresql://...
SUPABASE_URL=https://...supabase.co
SUPABASE_JWT_SECRET=...
OPENAI_API_KEY=sk-...
PORT=3001

# Supabase

# Frontend
NEXT_PUBLIC_SUPABASE_URL=https://...supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=...
NEXT_PUBLIC_API_URL=http://localhost:3001
```

9. Convenções de Código

- **pnpm** em todos os pacotes. Sem npm, sem yarn.
- **TypeScript estrito** em frontend e backend.
- **Zod** como fonte única da verdade para validação de entrada (HTTP, IA, forms).
- **Naming**: pastas em kebab-case; classes/types em PascalCase; variáveis e funções em camelCase.
- **Commits** em formato convencional (feat:, fix:, chore:, refactor:).
- **Branches**: main protegida; trabalho em feat/... ou fix/...; PR obrigatório.
- **Features não importam de outras features** — comunicação via core ou serviços explicitamente expostos.

10. O que NÃO entra agora

Lista explícita de coisas que **não** serão implementadas no MVP, para evitar retrabalho e debate desnecessário:

- Lógica real de **créditos** e **cobrança** (apenas o módulo placeholder).
- Integração com gateway de pagamento.
- Telas administrativas e dashboard de métricas internas (logs e queries diretas servem).
- Notificações por e-mail ou push.

- Aplicativo mobile.
- Internacionalização (apenas pt-BR).
- Provedores de login além do Google.
- Banco de questões público / explorável pelo usuário.

Resumo de uma frase

Next.js + Nest.js + Prisma/Postgres + OpenAI direto, com core/features no padrão MyMobiConf — simples o suficiente para 2 devs entregarem o MVP em semanas, não meses.